

Trabajo Practico Integrador

Universidad Tecnológica Nacional

Facultad Regional Mendoza

Tecnicatura Universitaria en Programación a distancia

Asignatura: Programación I

Integrantes: Ignacio Ortubia y Tiago Gusmano

Fecha de entrega: 20/6/2026

1. Caratula.....	1
2. Índice.....	2
3. Marco Teórico.....	3
Listas.....	3
Diccionarios.....	4
Funciones.....	5
Estructuras Condicionales y Repetitivas.....	6
Algoritmos de Ordenamiento.....	7
Procesamiento de Datos y Estadísticas Básicas.....	8
CSV.....	9
4. Decisiones técnicas y arquitectura.....	10
Arquitectura del sistema y modularización.....	10
Diagrama de flujo del menú.....	11
Verificación de funcionalidades.....	12
5. Dificultades y conclusiones.....	15
6. Bibliografía/webgrafía.....	16
7. Enlaces del proyecto.....	16

Marco teórico

Estructuras de datos lineales: Listas

En el ámbito de la ciencia de la computación, una estructura de datos lineal es aquella cuyos elementos se organizan de manera secuencial, donde cada componente de la estructura (a excepción del primero y el último) posee un único elemento predecesor y un único sucesor. Dentro del lenguaje de programación Python, la herramienta nativa por excelencia para implementar este concepto es la lista.

Una lista se define como una colección ordenada, mutable y dinámica de elementos, los cuales pueden ser de tipos de datos homogéneos o heterogéneos (enteros, cadenas de caracteres, flotantes o incluso otras estructuras complejas como diccionarios).

Orden: Cada elemento dentro de la lista ocupa una posición física fija y lineal que se encuentra asociada a un índice numérico entero. En Python, la indexación se basa en cero (0-indexed), lo que significa que el primer elemento de la colección se recupera mediante el índice 0.

Mutabilidad: A diferencia de las tuplas, las listas son estructuras mutables. Esto implica que es factible modificar, reemplazar, añadir o eliminar elementos en tiempo de ejecución sin necesidad de alterar la dirección de memoria de la estructura original (modificación in-place).

Dinamismo: Las listas en Python no requieren una declaración previa de su tamaño ni reservan un espacio estático en la memoria física. La asignación de memoria crece o decrece de forma automática a medida que el flujo del programa lo demanda.

Aplicación en el trabajo practico integrador:

Dentro del sistema desarrollado para la gestión de países, las listas constituyen la columna vertebral del almacenamiento y ordenamiento de los datos en la memoria volátil de la aplicación. Al invocar la función “carga_datos()”, el programa inicializa una lista maestra vacía llamada “países”

A medida que el archivo países.csv es leído mediante la clase csv.DictReader, el sistema utiliza el método nativo .append() para acoplar dinámicamente cada registro estructurado como diccionario dentro de dicha lista, consolidando una arquitectura de lista de diccionarios.

Esta estructura lineal es fundamental, ya que actúa como la colección indexada sobre la cual operan los algoritmos de ordenamiento (a través de la función integrada sorted()) y los mecanismos de comprensión de listas (list comprehensions) implementados para generar los vectores filtrados de resultados en las funciones buscar_pais(países) y filtrar_pais(países)

Estructuras de datos asociativas: Diccionarios

A diferencia de las estructuras lineales basadas en secuencias indexadas numéricamente, las estructuras de datos asociativas permiten establecer correspondencias directas entre elementos mediante vínculos semánticos. En el ecosistema de Python, la implementación nativa y más eficiente de este concepto se realiza a través de los diccionarios.

Un diccionario se define formalmente como una colección mutable, dinámica y no ordenada de pares clave-valor (en versiones contemporáneas de Python se preserva el orden de inserción, aunque conceptualmente su naturaleza sigue siendo asociativa). A nivel técnico, sus propiedades fundamentales incluyen:

Mapeo Clave-Valor: Cada elemento almacenado no se recupera mediante una posición posicional o índice entero (0, 1, 2...), sino invocando una "clave" (key). La clave actúa como el identificador único del dato y el "valor" (value) representa el contenido asociado a dicha clave.

Unicidad de Claves: Las claves dentro de un mismo diccionario deben ser estrictamente únicas e inmutables (típicamente cadenas de caracteres o enteros), lo que impide la duplicidad de identificadores. Los valores, por el contrario, pueden duplicarse y admiten cualquier tipo de dato, simple o complejo.

Eficiencia de Acceso: Los diccionarios están implementados internamente mediante tablas de dispersión (hash tables). Esto garantiza que las operaciones de búsqueda, inserción y modificación de un elemento se realicen en tiempo constante promedio, optimizando el rendimiento computacional sin importar el volumen de datos.

Aplicación en el Trabajo Practico Integrador:

En el sistema desarrollado, los diccionarios juegan un rol crítico en el modelado de las entidades del dominio: los países. Mientras que la lista maestra agrupa a todo el conjunto de naciones, cada país individual es representado internamente como un diccionario independiente. Esta decisión de diseño permite estructurar de forma clara y explícita las propiedades requeridas por la consigna.

La ventaja metodológica de aplicar diccionarios radica en la legibilidad y mantenibilidad del código: cuando las funciones de ordenamiento, filtrado o cálculo estadístico necesitan acceder, por ejemplo, a la cantidad de habitantes, no requieren recordar una posición de índice arbitraria, sino que invocan semánticamente la clave ['poblacion'], reduciendo drásticamente la probabilidad de errores humanos en la lógica del programa.

Modularización y diseño de software: Funciones

En la ingeniería de software, la modularización es la técnica de diseño que consiste en dividir un sistema de software complejo en partes más pequeñas, independientes y manejables llamadas módulos o subprogramas. En Python, esta abstracción se implementa a través de las funciones, definidas mediante la palabra clave `def`.

El diseño orientado a funciones se sustenta bajo principios metodológicos rigurosos:

Principio de Única Responsabilidad (Single Responsibility Principle): Establece que un bloque de código funcional debe realizar una única tarea conceptual específica y hacerla de manera eficiente.

Reutilización de Código (DRY - Don't Repeat Yourself): Evita la duplicidad de instrucciones en el sistema, permitiendo que una misma subrutina sea invocada múltiples veces desde diferentes secciones del flujo del programa.

Encapsulamiento y Abstracción: Oculta la complejidad interna del procesamiento de datos al programa principal. El bloque invocador solo necesita conocer los parámetros de entrada requeridos y el tipo de valor de retorno obtenido, abstrayéndose del cómo se ejecuta la tarea.

Aplicación en el Trabajo Practico Integrador:

El código fuente implementado para el sistema de gestión demuestra un uso avanzado de la modularización a través de una división limpia entre el flujo principal y la lógica de negocio.

Hay dos grandes categorías de funciones diseñadas bajo criterios académicos:

Funciones de Infraestructura y Validación: Subprogramas transversales como `leer_entero(mensaje)` y `leer_cadena(mensaje)`. Estas funciones encapsulan el control de excepciones por consola (`try-except ValueError`) y ciclos infinitos de validación. Al aplicarlas, se garantiza que ninguna otra función del sistema reciba datos corruptos, campos vacíos o valores numéricos negativos.

Funciones de Dominio o Negocio: Subprogramas especializados en los requerimientos del TPI como `agregar_pais(paises)`, `filtrar_pais(paises)` u `ordenar_paises(paises)`. El menú del programa principal delega el control absoluto a estas funciones pasándoles la lista maestra por parámetro, manteniendo el archivo ejecutable limpio y legible.

Asimismo, la inclusión de funciones auxiliares dedicadas a tareas específicas como `normalizar_texto(texto)` (diseñada exclusivamente para remover tildes y homogeneizar caracteres en minúsculas) e `imprimir_tabla(paises)` (especializada en el formateo visual de las salidas de datos en consola) ejemplifican una arquitectura modular desacoplada y escalable.

Control de flujo: Estructuras condicionales y repetitivas

El control de flujo determina el orden en el que se ejecutan las instrucciones de un programa informático basándose en condiciones lógicas o ciclos de iteración. En el lenguaje Python, estas estructuras permiten romper la ejecución secuencial (línea por línea) para dotar al software de dinamismo y capacidad de toma de decisiones.

Estructuras Condicionales (if, elif, else): Son bifurcaciones lógicas que evalúan una expresión booleana (verdadera o falsa). Si la condición se cumple, se ejecuta un bloque de código determinado; de lo contrario, el flujo se desvía hacia otras ramas alternativas.

Estructuras Repetitivas Indefinidas (while): Ejecutan un bloque de código continuamente "mientras" una condición lógica permanezca activa o verdadera. Es la estructura estándar para mantener programas interactivos abiertos.

Estructuras Repetitivas Definidas (for): Iteran sobre una colección o secuencia finita de elementos (como una lista o un rango numérico), ejecutando las instrucciones internas una vez por cada elemento presente en dicha colección.

Aplicación en el Trabajo Practico Integrador:

El sistema desarrollado implementa estas estructuras de control de flujo en dos niveles críticos de su arquitectura:

La Interfaz del Menú Principal: El programa utiliza un ciclo infinito controlado mediante la instrucción `while True`. Este bucle garantiza que el sistema permanezca en ejecución y muestre las opciones en consola constantemente, impidiendo que el programa finalice de forma prematura. Dentro de este ciclo, se acopla una estructura condicional múltiple `if-elif-else` que evalúa la variable `opcion` ingresada por el usuario. Cada rama (`elif`) invoca una función específica del sistema, mientras que la opción `'7'` rompe el ciclo mediante la instrucción `break` para permitir una salida limpia y controlada. La cláusula `else` final actúa como un capturador de errores ante opciones inválidas.

La Lógica de Filtrado y Recorrido: Los bucles tradicionales y los mecanismos de comprensión de listas recorren la lista de datos analizando los diccionarios de forma iterativa. Por ejemplo, en la función `actualizar_datos(paises)`, se aplica un bucle `for pais in paises` para examinar cada registro individual, combinado con un condicional `if normalizar_texto(pais["nombre"]) == normalizar_texto(nombre)`: encargado de hallar la coincidencia exacta antes de modificar los valores de población y superficie.

Algoritmos de ordenamiento

El ordenamiento de datos es una operación fundamental en la ciencia de la computación que consiste en reorganizar los elementos de una colección o estructura de datos siguiendo un criterio de relación de orden específico, que puede ser numérico (de menor a mayor o viceversa) o alfabético (en orden lexicográfico).

En el lenguaje Python, existen dos mecanismos principales para ordenar colecciones lineales de manera eficiente: el método mutable `.sort()` exclusivo de las listas, y la función integrada e inmutable `sorted()`. Esta última destaca por su versatilidad, ya que recibe cualquier objeto iterable por parámetro y devuelve una nueva lista con los elementos perfectamente ordenados, sin alterar la estructura original de los datos.

Para refinar el comportamiento de ordenamiento sobre estructuras complejas o anidadas, Python implementa dos parámetros clave:

El parámetro `key`: Permite pasar una función auxiliar que extrae un criterio de comparación para cada elemento. En desarrollos modernos, se suele combinar con funciones `lambda` (funciones anónimas de una sola línea), las cuales permiten especificar qué clave del diccionario interno debe evaluarse en tiempo de ejecución.

El parámetro `reverse`: Recibe un valor booleano (`True` o `False`). Si es verdadero, el algoritmo invierte el resultado, transformando un ordenamiento ascendente en uno descendente de forma inmediata.

Aplicación en el Trabajo Practico Integrador:

Dentro de la función `ordenar_paises(paises)` de nuestro sistema, se implementó la función integrada `sorted()` para resolver las necesidades de ordenamiento dinámico solicitadas por el usuario a través del menú de la consola.

Dado que la lista contiene diccionarios en su interior, fue indispensable emplear funciones anónimas `lambda` para indicarle al intérprete de Python sobre qué clave asociativa realizar las comparaciones. Dependiendo de la opción elegida por el operador, el sistema ejecuta una de las siguientes líneas lógicas:

Orden Alfabético: `sorted(paises, key=lambda x: x["nombre"], reverse=desendente)`

Orden por Población: `sorted(paises, key=lambda x: x["poblacion"], reverse=desendente)`

Orden por Superficie: `sorted(paises, key=lambda x: x["superficie"], reverse=desendente)`

La variable booleana `desendente` (definida previamente a partir de la entrada 'A' o 'D' del usuario) se asigna de manera directa al parámetro `reverse`. Esta arquitectura algorítmica permite que con un bloque mínimo de código limpio se logren cubrir seis flujos de ordenamiento distintos (tres criterios en sus variantes ascendente y descendente).

Procesamiento de datos y estadísticas básicas

El procesamiento de datos consiste en la recolección y manipulación de elementos de datos informáticos para producir información significativa y de valor estructurado. En el desarrollo de software, las operaciones de reducción y agregación estadística (como la obtención de valores extremos y promedios) representan las herramientas fundamentales para transformar registros individuales en indicadores clave de rendimiento (KPI).

Python provee funciones nativas optimizadas a nivel de lenguaje para realizar tareas de procesamiento masivo sobre iterables sin necesidad de implementar algoritmos manuales complejos:

Funciones de Extremos (max() y min()): Evalúan una colección y devuelven el elemento con el valor máximo o mínimo respectivamente. Al igual que los algoritmos de ordenamiento, admiten el parámetro key mediante funciones lambda para evaluar propiedades específicas dentro de estructuras anidadas.

Función de Acumulación (sum()): Calcula la sumatoria total de los elementos numéricos presentes en un iterable o generador de manera directa.

Estructuras de Frecuencia (Mapeos de Conteo): Consiste en utilizar la lógica de diccionarios para agrupar elementos por categorías, donde las claves representan las categorías únicas y los valores actúan como contadores acumulativos en tiempo de ejecución.

Aplicación en el Trabajo Practico Integrador:

Dentro de la función `mostrar_estadisticas(paises)`, se explotan al máximo estas funciones integradas de Python para calcular los indicadores solicitados por la cátedra, garantizando una ejecución eficiente y con un consumo mínimo de líneas de código:

Cálculo de Extremos: Para identificar geográficamente las naciones en los límites demográficos, el sistema ejecuta las instrucciones `max(paises, key=lambda x: x["poblacion"])` y `min(paises, key=lambda x: x["poblacion"])`. Esto permite extraer directamente el diccionario completo del país con mayor y menor cantidad de habitantes.

Cálculo de Promedios Globales: Se utiliza la función `sum()` alimentada por una expresión generadora para calcular los totales absolutos de habitantes y superficie (`sum(p["poblacion"] for p in paises)`). Posteriormente, el promedio aritmético se obtiene dividiendo dicha sumatoria por la cantidad total de registros, calculada mediante la función nativa `len(paises)`.

Distribución por Continente: Para contabilizar los países por región, se inicializa un diccionario vacío llamado `conteo_continentes`. El sistema recorre la lista maestra y, mediante una estructura condicional, verifica si el continente ya existe como clave; si existe, incrementa su valor en uno (`+= 1`), y si no, lo inicializa en uno (`= 1`).

Finalmente, los resultados numéricos se exponen al usuario formateados mediante f-strings empleando especificadores de formato avanzados (como `;` para separar miles por comas y `:.2f`

para truncar los promedios a dos decimales flotantes), logrando una salida visual profesional en la consola.

Persistencia de datos: Archivos de valores separados por comas (CSV)

En el desarrollo de software, la persistencia de datos hace referencia a la propiedad de la información de sobrevivir a la finalización del proceso que la creó. Sin mecanismos de persistencia, todos los datos modificados o generados por un programa se perderían de forma irreversible al cerrarse la aplicación o apagarse el equipo, dado que residen únicamente en la memoria RAM (memoria volátil).

Para lograr persistencia sin la complejidad de un motor de bases de datos relacionales, se emplean archivos de texto plano, siendo el formato CSV (Comma-Separated Values) uno de los estándares abiertos más utilizados en la industria para el intercambio de datos tabulares. Un archivo CSV organiza la información en líneas (filas), donde cada dato específico (columna) se separa formalmente mediante un carácter delimitador, típicamente una coma (,) o un punto y coma (;).

Para interactuar de manera segura con el sistema de archivos del sistema operativo, Python provee dos recursos nativos clave:

El módulo `os`: Permite interactuar con las rutas y directorios del disco físico, ofreciendo funciones críticas para verificar de manera preventiva la existencia de recursos antes de lanzar operaciones de lectura.

El módulo `csv`: Provee clases de alto nivel como `DictReader` y `DictWriter`. Estas herramientas automatizan el mapeo directo entre las filas de texto del archivo y las estructuras asociativas de diccionarios en Python, abstrayendo al programador de la manipulación manual de cadenas.

Aplicación en el Trabajo Practico Integrador:

El sistema de gestión de países garantiza la persistencia total de los datos mediante dos flujos simétricos y desacoplados programados en `funciones.py`:

Lectura y Recuperación (`carga_datos()`): Al iniciar la aplicación, la función utiliza la instrucción condicional `os.path.exists(archivo_csv)` para validar si el archivo físico está presente en el directorio. Si existe, se abre de forma segura utilizando el administrador de contexto `with open()`. Internamente, la clase `csv.DictReader(archivo)` interpreta de forma automática la primera línea como las claves del diccionario (nombre, continente, etc.) y procede a deserializar las filas de texto, permitiendo realizar la conversión de tipos necesaria (`int()`) para la población y la superficie antes de incorporar los registros a la lista interna.

Escritura y Almacenamiento (`guardar_datos(paises)`): Cada vez que el estado de la lista se altera (ya sea por la inserción de una nueva nación en `agregar_pais` o por la edición de valores en `actualizar_datos`), el sistema invoca de forma automática la persistencia de salida. Utilizando `csv.DictWriter(archivo, fieldnames=campos)`, se sobrescribe el archivo externo guardando la cabecera estructurada mediante `.writeheader()` y volcando la lista completa de diccionarios en un solo bloque con el método `.writerows(paises)`.

Decisiones Técnicas y Arquitectura

Arquitectura del sistema y modularización

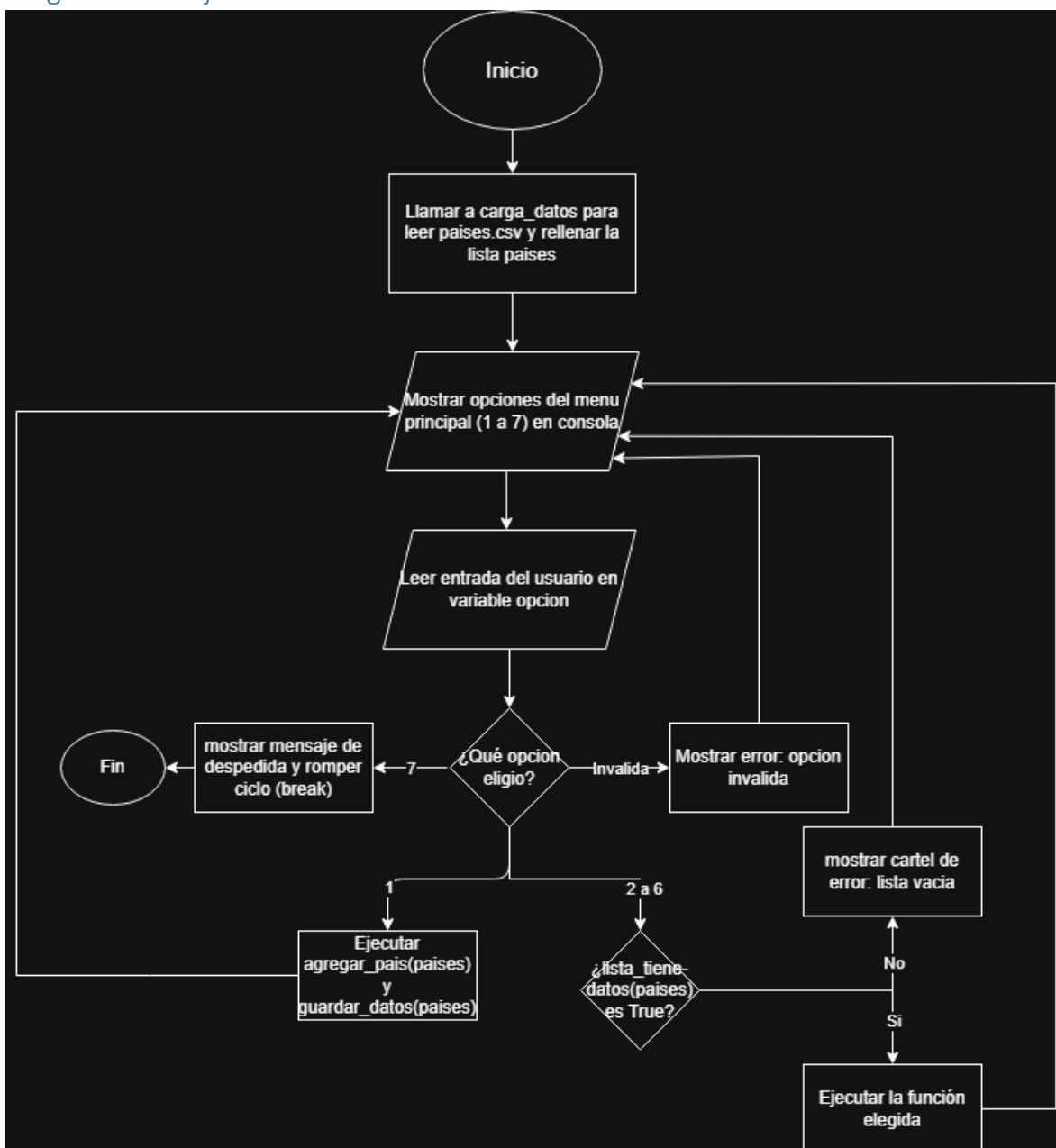
El diseño de la aplicación se estructuró bajo un criterio de separación de responsabilidades, dividiendo el sistema en dos componentes de software claramente definidos para facilitar su mantenimiento y escalabilidad:

Módulo de Interfaz (main.py): Actúa como el punto de entrada principal de la aplicación. Su única responsabilidad es gestionar el ciclo de vida del menú en consola, capturar la opción del operador e invocar a la subrutina correspondiente. No contiene lógica de negocio ni manipula directamente los datos.

Módulo de Lógica y Persistencia (funciones.py): Concentra la totalidad de las funciones del sistema. Centraliza las tareas de lectura/escritura del archivo `países.csv`, las validaciones de entrada de datos (try-except) y los algoritmos analíticos (búsquedas, filtros, ordenamientos y estadísticas).

Esta arquitectura desacoplada garantiza que cualquier modificación en la interfaz gráfica o en el formato de almacenamiento de los datos pueda realizarse afectando a un único archivo, reduciendo el riesgo de introducir errores colaterales en el resto del software.

Diagrama de flujo del menú



Verificación de funcionalidades y evidencia de ejecución

Menú principal y carga inicial

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - [ ] [ ] ... | [ ] [ ] X
PS C:\Users\Usuario\Documents\GitHub\Trabajo-Pr-ctico-Integrador-TPI-> & C:/Users/Usuario/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/Usuario/Documents/GitHub/Trabajo-Pr-ctico-Integrador-TPI-/Trabajo Práctico Integrador (TPI) main.py"

=====
SISTEMA DE GESTIÓN DE PAÍSES
=====
1. Agregar un país
2. Actualizar población y superficie
3. Buscar un país
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Salir

Seleccione una opción:
```

Opcion 1 – Agregar un pais

```
Seleccione una opción: 1
Ingrese el nombre del país: Argelia

Seleccione un continente:
1. America
2. Europa
3. Asia
4. Africa
5. Oceania
Ingrese el número del continente: 4
Ingrese la población del país: 20000000
Ingrese la superficie del país (en km²): 400000
País 'Argelia' agregado exitosamente.
```

Opcion 2 – Actualizar poblacion y superficie

```
Seleccione una opción: 2
Ingrese el nombre del país a actualizar: Argelia
Actualizando datos para Argelia:
Ingrese la nueva población del país: 10000000
Ingrese la nueva superficie del país (en km²): 300000
Datos de 'Argelia' actualizados exitosamente.
```

Opcion 3 – Buscar un país

```
Seleccione una opción: 3

--- BUSCAR PAÍS ---
Ingrese el nombre a buscar (parcial o exacto): arg

-----
NOMBRE          | POBLACIÓN      | SUPERFICIE     | CONTINENTE
-----
Argentina       | 47000000       | 2780400        | América
Argelia         | 10000000       | 300000         | africa
-----
```

Opcion 4 – Filtrar países

```
Seleccione una opción: 4

--- FILTRAR PAÍSES ---
1. Por Continente
2. Por Rango de Población
3. Por Rango de Superficie
Elija una opción de filtro: 1

  Seleccione un continente:
  1. America
  2. Europa
  3. Asia
  4. Africa
  5. Oceania
  Ingrese el número del continente: 1

-----
NOMBRE          | POBLACIÓN      | SUPERFICIE     | CONTINENTE
-----
Argentina       | 47000000       | 2780400        | América
Canadá          | 41000000       | 9984670        | América
Brasil          | 216000000      | 8515767        | América
-----
```

Opcion 5 – Ordenar paises

```
Seleccione una opción: 5

--- ORDENAR PAÍSES ---
1. Por Nombre (Alfabético)
2. Por Población
3. Por Superficie
Elija un criterio: 1
¿Orden Ascendente (A) o Descendente (D)?: a

-----
NOMBRE          | POBLACIÓN      | SUPERFICIE     | CONTINENTE
-----
Alemania         | 84000000       | 357022         | Europa
Argelia          | 10000000       | 300000         | africa
Argentina        | 47000000       | 2780400        | América
Australia        | 27000000       | 7692024        | Oceanía
Brasil           | 216000000      | 8515767        | América
Canadá           | 41000000       | 9984670        | América
China            | 1410000000     | 9596961        | Asia
Egipto           | 112000000      | 1002450        | África
España           | 49000000       | 505990         | Europa
India            | 1430000000     | 3287263        | Asia
Nigeria         | 227000000      | 923768         | África
-----
```

Opcion 6 – mostrar estadísticas

```
Seleccione una opción: 6

--- ESTADÍSTICAS ---
País con MAYOR población: India (1,430,000,000)
País con MENOR población: Argelia (10,000,000)
Promedio de población global: 332,090,909.09
Promedio de superficie global: 4,086,028.64 km²

Cantidad de países por continente:
- América: 3
- Europa: 2
- Asia: 2
- África: 2
- Oceanía: 1
- africa: 1
```

Opcion 7 – Salir

```
=====
SISTEMA DE GESTIÓN DE PAÍSES
=====
1. Agregar un país
2. Actualizar población y superficie
3. Buscar un país
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Salir

Seleccione una opción: 7

Guardando datos y saliendo del sistema. ¡Hasta luego!
PS C:\Users\Usuario\Documents\GitHub\Trabajo-Pr-ctico-Integrador-TPI-> |
```

Dificultades y conclusiones

Durante la fase inicial del ciclo de vida del proyecto, el equipo enfrentó un desfase crítico en la planificación debido a una subestimación de los requisitos de persistencia de datos solicitados en las consignas de la cátedra.

El desarrollo de la lógica del sistema se inició de forma prematura sin haber abordado previamente el estudio teórico-práctico de la unidad académica correspondiente al manejo de archivos planos. Al avanzar sobre una arquitectura basada puramente en estructuras volátiles y percatarnos de la obligatoriedad de integrar el almacenamiento persistente mediante el formato CSV desde el origen, el código desarrollado hasta ese momento quedó obsoleto por su falta de acoplamiento con el sistema de archivos.

Esta situación obligó al equipo a pausar la producción del software y reiniciar el diseño de la arquitectura desde cero, lo que generó un retraso imprevisto de aproximadamente dos semanas en el cronograma estipulado. Este tiempo debió reasignarse exclusivamente a la investigación, lectura de documentación técnica y asimilación de los conceptos de los módulos csv y os.

La realización de este Proyecto Integrador permitió consolidar de manera práctica los fundamentos de la programación estructurada y el diseño algorítmico adquiridos a lo largo del cursado. Más allá de la resolución técnica de los requerimientos, la experiencia evidenció que la ingeniería de software requiere una fase rigurosa de análisis de requisitos antes de proceder a la escritura de código, lección aprendida de forma directa a raíz de las dificultades experimentadas con la persistencia de datos.

La implementación final del "Sistema de Gestión de Países" demostró que el lenguaje Python ofrece herramientas nativas altamente eficientes para el procesamiento masivo y análisis estadístico de datos tabulares a través de una sintaxis limpia. Conceptos abstractos como las funciones lambda, el control de excepciones y la manipulación de estructuras anidadas (listas de

diccionarios) dejaron de ser meros bloques teóricos para transformarse en soluciones lógicas a problemas reales de automatización. El proyecto concluye con un software completamente funcional, escalable, robusto ante errores de operador y alineado con los estándares académicos exigidos.

Bibliografía/webgrafía

<https://docs.python.org/es/3.14/tutorial/datastructures.html>

<https://peps.python.org/pep-0008/>

<https://docs.python.org/3/howto/sorting.html>

https://www.w3schools.com/python/python_lambda.asp

<https://www.geeksforgeeks.org/python/python-sorted-function/>

<https://docs.python.org/3/library/csv.html>

<https://docs.python.org/3/library/os.path.html>

Enlaces del proyecto

Video demostrativo: <https://youtu.be/8NGoQ5YpfQ4> (copiar y pegar en el navegador)

Repositorio del proyecto: <https://github.com/ThiagoGusmano/Trabajo-Pr-ctico-Integrador-TPI->